```
// ============ 文件: lib/main.dart ============
import 'dart:async';

import 'package:amap_map/amap_map.dart';
import 'package:flutter/material.dart';
import 'package:x_amap_base/x_amap_base.dart';
import 'package:flutter/services.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'data/database.dart';
import 'data/repositories/asset_repository.dart';
import 'data/repositories/asset_flow_view_repository.dart';
import 'data/repositories/bill_image_repository.dart';
import 'data/repositories/bill_repository.dart';
import 'data/repositories/budget_repository.dart';
import 'data/repositories/category_repository.dart';
import 'data/repositories/debt_note_repository.dart';
import 'data/repositories/tag_repository.dart';
import 'pages/db_error_page.dart';
import 'pages/lock/lock_screen.dart';
import 'pages/main_page.dart';
import 'pages/privacy/privacy_consent_page.dart';
import 'providers/asset_provider.dart';
import 'providers/bill_provider.dart';
import 'providers/budget_provider.dart';
import 'providers/category_provider.dart';
import 'providers/cloud_storage_provider.dart';
import 'providers/debt_note_provider.dart';
import 'providers/image_sync_status_provider.dart';
import 'providers/lock_provider.dart';
import 'providers/settings_provider.dart';
import 'services/backup_service.dart';
import 'services/photo_staging_service.dart';
import 'theme/app_theme.dart';

/// 隐私政策同意标记：未同意前不初始化高德等第三方 SDK、不进入主界面
const String kPrivacyAgreedKey = 'privacy_policy_agreed';

/// 向高德 SDK 声明隐私状态（政策已包含、已弹窗、已同意）。
/// 必须在任何地图组件创建前调用，否则地图白屏且不合规。
void _declareAmapPrivacyAgreed() {
  AMapInitializer.updatePrivacyAgree(
    const AMapPrivacyStatement(hasContains: true, hasShow: true, hasAgree: true),
  );
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
```

```
  // 固定竖屏，记账场景无横屏需求（不需等待生效，不阻塞启动）
  unawaited(
    SystemChrome.setPreferredOrientations([DeviceOrientation.portraitUp]),
  );
  // 启动早期执行待定数据恢复：必须在数据库首次打开前完成文件替换
  await BackupService().restoreIfNeeded();
  // 清理上次会话遗留的导出临时文件（分享时微信异步读取，不能当场删）
  await BackupService().cleanupExportTemp();
  // 预加载设置存储，UI 各处可同步读取开关状态
  final prefs = await SharedPreferences.getInstance();
  // 已同意过隐私政策：启动即完成高德 SDK 合规声明（首启由同意页触发）
  if (prefs.getBool(kPrivacyAgreedKey) ?? false) {
    _declareAmapPrivacyAgreed();
  }
  // 只读健康探测：主库损坏时走兜底页，避免白屏或崩溃
  final dbHealthy = await BackupService().checkDatabaseHealth();
  // 每日自动备份：用户开启开关后每天首次启动执行（库不健康时跳过）
  if (dbHealthy) await BackupService().autoBackupIfNeeded(prefs);
  // 清理图片暂存目录：上次会话选了图但没保存的残留文件
  await PhotoStagingService(dirName: 'receipts').cleanupStaging();
  // 清理借条照片暂存目录：同理（借据照片选了没保存的残留）
  await PhotoStagingService(dirName: 'debt_photos').cleanupStaging();
  runApp(ChestnutApp(prefs: prefs, dbHealthy: dbHealthy));
}

/// 应用根组件：主库健康时进入依赖注入与主页，损坏时展示兜底页
class ChestnutApp extends StatelessWidget {
  const ChestnutApp({super.key, required this.prefs, this.dbHealthy = true});

  final SharedPreferences prefs;

  /// 主库健康探测结果；false 时展示数据库错误兜底页，
  /// 不注入数据库依赖，避免打开损坏库引发更严重的错误
  final bool dbHealthy;

  @override
  Widget build(BuildContext context) {
    return dbHealthy
        ? _ProvidersApp(prefs: prefs)
        : const _MaterialShell(home: DatabaseErrorPage());
  }
}

/// MaterialApp 公共壳：中文本地化 + 主题，主页与兜底页两处复用
class _MaterialShell extends StatelessWidget {
  const _MaterialShell({required this.home, this.lockGate = false});

  final Widget home;

  /// 是否注入密码锁遮罩：仅依赖注入壳启用；
```

```
  /// 兜底页无 provider 且数据库已损坏，锁屏无意义
  final bool lockGate;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: '栗子记账',
      debugShowCheckedModeBanner: false,
      // 中文本地化：钟面时间选择器、日期选择器等内置控件显示中文
      localizationsDelegates: const [
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      supportedLocales: const [Locale('zh'), Locale('en')],
      locale: const Locale('zh'),
      theme: AppTheme.light,
      home: home,
      // 锁定遮罩挂在 builder 上：位于 Navigator 之上，任何页面都会被盖住
      builder: lockGate
          ? (context, child) =>
                _LockGate(child: child ?? const SizedBox.shrink())
          : null,
    );
  }
}

/// 密码锁遮罩：启用密码且处于锁定态时全屏盖锁屏页；
/// Stack 保留下层 Navigator，解锁瞬间无需整树重建
class _LockGate extends StatelessWidget {
  const _LockGate({required this.child});

  final Widget child;

  @override
  Widget build(BuildContext context) {
    final needLock = context.watch<SettingsProvider>().passcodeEnabled &&
        context.watch<LockProvider>().locked;
    return Stack(
      children: [
        child,
        if (needLock) const Positioned.fill(child: LockScreen()),
      ],
    );
  }
}

/// 依赖注入容器：数据库 → 仓储 → Provider → 主页
///
/// MultiProvider 必须包在 MaterialApp 之上：push 出的二级页面是
```

```
/// Navigator 里与 home 平级的兄弟 OverlayEntry，provider 若注入在
/// home 内部，二级页面沿树向上找不到，会报 Provider not found。
class _ProvidersApp extends StatelessWidget {
  const _ProvidersApp({required this.prefs});

  final SharedPreferences prefs;

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        // 数据库为全局单例，应用退出时由 dispose 关闭
        Provider<AppDatabase>(
          create: (_) => AppDatabase(),
          dispose: (_, db) => db.close(),
        ),
        // 仓储层：一次性创建的单例依赖，用 read 获取上游实例即可
        Provider<BillRepository>(
          create: (ctx) => BillRepository(ctx.read<AppDatabase>()),
        ),
        Provider<CategoryRepository>(
          create: (ctx) => CategoryRepository(ctx.read<AppDatabase>()),
        ),
        Provider<BudgetRepository>(
          create: (ctx) => BudgetRepository(ctx.read<AppDatabase>()),
        ),
        Provider<TagRepository>(
          create: (ctx) => TagRepository(ctx.read<AppDatabase>()),
        ),
        Provider<BillImageRepository>(
          create: (ctx) => BillImageRepository(ctx.read<AppDatabase>()),
        ),
        Provider<AssetRepository>(
          create: (ctx) => AssetRepository(ctx.read<AppDatabase>()),
        ),
        Provider<DebtNoteRepository>(
          create: (ctx) => DebtNoteRepository(ctx.read<AppDatabase>()),
        ),
        Provider<AssetFlowViewRepository>(
          create: (ctx) =>
              AssetFlowViewRepository(ctx.read<AppDatabase>()),
        ),
        // 图片云存储配置：未配置/未启用时 App 内不出现任何图片入口。
        // 必须注册在 BillProvider 之前——MultiProvider 列表前面的包住
        // 后面的（祖先方向），BillProvider 的 create 要 read 它
        ChangeNotifierProvider<CloudStorageProvider>(
          create: (_) => CloudStorageProvider(prefs),
        ),
        // 状态层
        ChangeNotifierProvider<BillProvider>(
```

```
          create: (ctx) => BillProvider(
            ctx.read<BillRepository>(),
            ctx.read<BillImageRepository>(),
            ctx.read<CloudStorageProvider>(),
          ),
        ),
        ChangeNotifierProvider<CategoryProvider>(
          create: (ctx) => CategoryProvider(ctx.read<CategoryRepository>()),
        ),
        ChangeNotifierProvider<BudgetProvider>(
          create: (ctx) => BudgetProvider(
            ctx.read<BudgetRepository>(),
            ctx.read<BillRepository>(),
          ),
        ),
        ChangeNotifierProvider<AssetProvider>(
          create: (ctx) =>
              AssetProvider(ctx.read<AssetRepository>(), prefs),
        ),
        ChangeNotifierProvider<DebtNoteProvider>(
          create: (ctx) => DebtNoteProvider(ctx.read<DebtNoteRepository>()),
        ),
        // 图片云同步状态：聚合小票/借据两表计数，同步页与两处红点角标共用
        ChangeNotifierProvider<ImageSyncStatusProvider>(
          create: (ctx) => ImageSyncStatusProvider(
            ctx.read<BillImageRepository>(),
            ctx.read<DebtNoteRepository>(),
          ),
        ),
        ChangeNotifierProvider<SettingsProvider>(
          create: (_) => SettingsProvider(prefs),
        ),
        // 密码锁：读取设置开关与超时配置，冷启动即决定是否进入锁定态
        ChangeNotifierProvider<LockProvider>(
          create: (ctx) => LockProvider(ctx.read<SettingsProvider>()),
        ),
      ],
      child: _MaterialShell(home: _StartupGate(prefs: prefs), lockGate: true),
    );
  }
}

/// 启动门：首次启动未同意隐私政策时只显示同意页（MainPage 不构建，
/// 因此首页的更新检测等网络行为在同意前均不会发生）；同意后切换进主页
class _StartupGate extends StatefulWidget {
  const _StartupGate({required this.prefs});

  final SharedPreferences prefs;

  @override
```

```
  State<_StartupGate> createState() => _StartupGateState();
}

class _StartupGateState extends State<_StartupGate> {
  late bool _agreed =
      widget.prefs.getBool(kPrivacyAgreedKey) ?? false;

  /// 补传进行中标记：冷启动与回前台的多次触发互斥，避免并发重复上传
  bool _uploadingImages = false;

  AppLifecycleListener? _lifecycleListener;

  @override
  void initState() {
    super.initState();
    // 启动静默补传（仅同意隐私政策后，与其他联网行为同口径）：
    // 扫描 uploadState != done 的图片记录逐张补传，失败留在下次启动重试
    if (_agreed) unawaited(_uploadPendingImages());
    // 回前台也触发补传：App 只是切到后台未死时（如记账后切微信再切回），
    // 冷启动补传不会发生，回到前台时同样静默补一次
    _lifecycleListener = AppLifecycleListener(
      onResume: () => unawaited(_uploadPendingImages()),
    );
  }

  @override
  void dispose() {
    _lifecycleListener?.dispose();
    super.dispose();
  }

  /// 延迟触发静默补传：避开启动高峰，也让弱网下首屏请求优先。
  /// 未配置/未启用云存储时 createClient 为 null，无任何图片网络行为
  Future<void> _uploadPendingImages() async {
    if (_uploadingImages) return;
    _uploadingImages = true;
    try {
      // 捕获依赖后再进入 await：补传是异步任务，不能依赖 async gap 后的 context
      final repo = context.read<BillImageRepository>();
      final debtRepo = context.read<DebtNoteRepository>();
      await Future<void>.delayed(const Duration(seconds: 5));
      if (!mounted) return;
      final client = context.read<CloudStorageProvider>().createClient();
      if (client == null) return;
      await repo.uploadPending(client);
      // 借据照片同口径补传（失败的单张由下次触发继续重试）
      await debtRepo.uploadPending(client);
    } finally {
      _uploadingImages = false;
    }
```

```
  }

  Future<void> _agree() async {
    await widget.prefs.setBool(kPrivacyAgreedKey, true);
    _declareAmapPrivacyAgreed();
    if (mounted) setState(() => _agreed = true);
  }

  @override
  Widget build(BuildContext context) {
    return _agreed
        ? const MainPage()
        : PrivacyConsentPage(onAgree: _agree);
  }
}
// ============ 文件: lib/data/database.dart ============
import 'dart:io';

import 'package:drift/drift.dart';
import 'package:drift/native.dart';
import 'package:flutter/material.dart' show Icons, IconData;
import 'package:intl/intl.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:sqlite3/sqlite3.dart';

import '../models/enums.dart';
import '../models/institutions.dart';
import 'tables/assets.dart';
import 'tables/bill_images.dart';
import 'tables/bills.dart';
import 'tables/budgets.dart';
import 'tables/categories.dart';
import 'tables/debt_note_images.dart';
import 'tables/debt_notes.dart';
import 'tables/tags.dart';

part 'database.g.dart';

/// 当前数据库结构版本
///
/// 开发期历史迁移已在发布前整体重置归一，自 v1 起每次结构
/// 变更 +1；野外用户出现后版本号只增不减、迁移代码只增不删。
const int kSchemaVersion = 13;

/// 数据库主文件名（备份服务与启动恢复共用）
const String kDatabaseFileName = 'chestnut.sqlite';

/// WAL 模式附属文件：替换主库文件时必须一并删除，否则旧 WAL 内容
```

```
/// 会叠加到新主文件上导致数据库损坏
const List<String> kDatabaseSidecarFiles = [
  'chestnut.sqlite-wal',
  'chestnut.sqlite-shm',
];

/// 降级发生标记（SharedPreferences key）
///
/// 降级重建后 App 内数据被清空，用户极易误以为数据全丢；首页启动
/// 后检测此标记弹恢复引导，读取后即清除（只弹一次）。
const String kDowngradeDetectedKey = 'db_downgrade_detected';

/// 应用数据库
///
/// 单例式入口：负责建库、迁移与首次预置默认分类。
@DriftDatabase(tables: [
  Categories,
  Bills,
  Budgets,
  Tags,
  BillTags,
  BillImages,
  Assets,
  AssetSnapshots,
  DebtNotes,
  DebtNoteImages,
])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(_openConnection(kSchemaVersion));

  /// 仅测试使用：注入内存执行器，不落盘
  AppDatabase.forTesting(super.executor);

  @override
  int get schemaVersion => kSchemaVersion;

  @override
  MigrationStrategy get migration => MigrationStrategy(
    onCreate: (m) async {
      await m.createAll();
      // 建库时预置内置分类（含子分类），避免新用户面对空分类列表
      await _seedDefaultCategories();
    },
    // 升级与降级统一在此处理（drift 把升降级都送进 onUpgrade）：
    // from > to 即降级——用户装回旧版 App，旧代码无法识别新结构，
    // drift 默认抛异常导致启动崩溃死循环，这里改为重建空库保证可用，
    // 高版本数据已在打开连接前由 _backupBeforeOpen 留底。
    // from < to 的正常升级走 else 分支，逐版本追加列迁移。
    onUpgrade: (m, from, to) async {
      if (from > to) {
```

```
      // 删除所有实体（表/索引/触发器）后按当前代码结构重建
      for (final entity in allSchemaEntities) {
        await m.drop(entity);
      }
      await m.createAll();
      await _seedDefaultCategories();
      // 标记降级已发生，首页据此弹恢复引导（留底文件已在打开前生成）
      try {
        final prefs = await SharedPreferences.getInstance();
        await prefs.setBool(kDowngradeDetectedKey, true);
      } catch (_) {
        // 标记失败只影响提示，不影响重建结果
      }
    } else {
      // 正常升级：迁移只增不删。可空列直接 ADD COLUMN，
      // 旧账单优惠额为 null（未使用优惠）
      if (from < 2) {
        await m.addColumn(bills, bills.discountCents);
      }
      // 钱迹导入：批次号列（可空，手动记账与旧数据均为 null）
      if (from < 3) {
        await m.addColumn(bills, bills.importBatchId);
      }
      // 标签功能：新增 tags 与 bill_tags 两表（多对多关联）
      if (from < 4) {
        await m.createTable(tags);
        await m.createTable(billTags);
      }
      // 账单图片：新增 bill_images 表（小票/发票凭证）。
      // 注意 @TableIndex 声明的索引不随 createTable 创建，需显式建索引
      if (from < 5) {
        await m.createTable(billImages);
        await m.createIndex(billImagesBill);
      }
      // 资产管理：新增 assets 与 asset_snapshots 两表（净资产盘点）。
      // 索引同理需随迁移显式创建
      if (from < 6) {
        await m.createTable(assets);
        await m.createTable(assetSnapshots);
        await m.createIndex(assetSnapshotsAssetDay);
      }
      // 资产管理 v2（钱迹式）：assets 加净值开关与信用卡字段，
      // 新增借条表 debt_notes；旧资产分类名映射到新分类体系。
      // 全部做幂等保护：若上次迁移半途中断（列已加、版本号未落），
      // 重复 ADD COLUMN 会报 duplicate column 导致 App 不可用
      if (from < 7) {
        await _ensureColumn(m, assets, assets.includeInNet);
        await _ensureColumn(m, assets, assets.creditLimitCents);
        await _ensureColumn(m, assets, assets.billDay);
        await _ensureColumn(m, assets, assets.repayDay);
```

```
  if (!await _tableExists('debt_notes')) {
    await m.createTable(debtNotes);
  }
  await _migrateLegacyAssetCategories();
}
// 借据照片云端双写：debt_notes 补云端对象键与上传状态列。
// 幂等加列（原生 SQL：这三列已从 drift 表定义移除，
// v10 借据多图改造后不再由 drift 管理）
if (from < 8) {
  await _ensureRawColumn(
    'debt_notes',
    'object_key',
    'object_key TEXT NULL',
  );
  await _ensureRawColumn(
    'debt_notes',
    'upload_state',
    'upload_state INTEGER NOT NULL DEFAULT 0',
  );
}
// 记账联动账户：bills 加 assetId/toAssetId 两列，categoryId 改可空
// （转账无分类语义）。SQLite 无法直接改列约束，用 TableMigration
// 重建 bills 表（索引随迁移一并重建）；加列走幂等 _ensureColumn。
// 注意 categoryId 不能放 newColumns——那是"旧表不存在的新列"语义，
// 复制数据时会被填 NULL 而非拷贝旧值；空迁移即按新定义重建并复制全部旧列
if (from < 9) {
  await _ensureColumn(m, bills, bills.assetId);
  await _ensureColumn(m, bills, bills.toAssetId);
  await m.alterTable(
    // ignore: experimental_member_use
    TableMigration(bills),
  );
}
// 借据照片改多图：新增 debt_note_images 表（与 bill_images 同构）。
// 先把旧单图三列（photo_path/object_key/upload_state）的数据迁入
// 新表，再重建 debt_notes 去掉这三列（SQLite 无法直接删列）。
// 注意 TableMigration 按新旧表同名列复制数据，借条主键 id 原样保留
if (from < 10) {
  await m.createTable(debtNoteImages);
  await m.createIndex(debtNoteImagesNote);
  if (await _tableExists('debt_notes')) {
    // 旧结构才有单图列（当前代码建的表已无 photo_path，直接跳过）
    final noteCols = (await customSelect(
      'PRAGMA table_info([debt_notes])',
    ).get()).map((r) => r.data['name'] as String).toSet();
    if (noteCols.contains('photo_path')) {
      final legacy = await customSelect(
        'SELECT id, photo_path, object_key, upload_state '
        'FROM debt_notes WHERE photo_path IS NOT NULL',
      ).get();
```

```
        for (final row in legacy) {
          final state = (row.data['upload_state'] as int?) ?? 0;
          await into(debtNoteImages).insert(
            DebtNoteImagesCompanion.insert(
              noteId: row.data['id'] as int,
              objectKey: (row.data['object_key'] as String?) ?? '',
              localPath: row.data['photo_path'] as String,
              uploadState: BillImageUploadState.values[state],
            ),
          );
        }
      }
      await m.alterTable(
        // ignore: experimental_member_use
        TableMigration(debtNotes),
      );
    } else {
      // 极端情况兜底：库中缺 debt_notes（如测试最小骨架）按新定义补建
      await m.createTable(debtNotes);
    }
  }
  // 所属机构独立成列：assets 加可空 institution（银行卡/信用卡=
  // 所属银行，投资理财组=券商/平台，名单见 institutions.dart）。
  // 旧数据中账户名恰好等于已知机构名（如"招商银行"、"华泰证券"）的，
  // 精准回填；带后缀（"招商银行卡"）或对不上的一律留空不硬猜，
  // 避免把用户自定义名称误判为机构
  if (from < 11) {
    await _ensureColumn(m, assets, assets.institution);
    final placeholders = List.filled(kAllInstitutionNames.length, '?')
        .join(',');
    await customUpdate(
      "UPDATE assets SET institution = name "
      "WHERE category IN ('信用卡', '银行卡', '股票', '基金', '黄金', "
      "'外汇', '期货', '债券', '固定收益', '加密货币', '其它理财') "
      "AND name IN ($placeholders)",
      variables: [
        for (final n in kAllInstitutionNames) Variable(n),
      ],
    );
  }
  // 开发期 v11 中间版曾给 assets 加过 bank 列，之后列改名 institution，
  // 而装过中间版的库版本号已是 11、不会再重跑 v11 段——institution
  // 列缺失导致保存报 no such column。v12 幂等收尾：bank 列存在则
  // 把值搬进 institution 并删掉 bank；正常 v10 直升的库无 bank 列，
  // 整段跳过
  if (from < 12) {
    // 中间态库只有 bank 列、没有 institution 列，先补齐再搬运
    await _ensureColumn(m, assets, assets.institution);
    final bankCol = await customSelect(
      "SELECT COUNT(*) AS c FROM pragma_table_info('assets') "
```

```
          "WHERE name = 'bank'",
        ).getSingle();
        if ((bankCol.data['c'] as int) > 0) {
          await customUpdate(
            'UPDATE assets SET institution = bank '
            'WHERE institution IS NULL AND bank IS NOT NULL',
          );
          try {
            await customUpdate('ALTER TABLE assets DROP COLUMN bank');
          } catch (_) {
            // 旧 SQLite（<3.35）不支持 DROP COLUMN：bank 留作空列，
            // drift 的读写不经过它，不影响功能
          }
        }
      }
      // 兜底分类标记：categories 加 isFallback 列（默认 false），
      // 并为老库补齐/打标收支「其他」——分类删除迁移与钱迹导入的
      // 共同承接分类，按标记识别（用户改名不失效）。幂等加列，
      // 已标记的分类跳过
      if (from < 13) {
        await _ensureColumn(m, categories, categories.isFallback);
        await _seedFallbackCategories();
      }
    }
  },
);

/// 预置默认分类（收支各一套，部分一级分类带子分类）
Future<void> _seedDefaultCategories() async {
  for (final seed in _defaultCategories) {
    final parentId = await into(categories).insert(
      CategoriesCompanion.insert(
        name: seed.name,
        iconCode: seed.icon.codePoint,
        colorValue: seed.color,
        type: seed.type,
        isFallback: Value(seed.isFallback),
        sortOrder: Value(seed.sortOrder),
      ),
    );
    final childRows = <CategoriesCompanion>[];
    for (var i = 0; i < seed.children.length; i++) {
      final child = seed.children[i];
      // 子分类颜色跟随父分类，保证饼图归并到一级时同色
      childRows.add(
        CategoriesCompanion.insert(
          name: child.name,
          iconCode: child.icon.codePoint,
          colorValue: child.color,
          type: child.type,
```

```
            parentId: Value(parentId),
            sortOrder: Value(i),
          ),
        );
      }
      if (childRows.isNotEmpty) {
        // drift 无单条 insertAll，批量插入统一走 batch
        await batch((b) => b.insertAll(categories, childRows));
      }
    }
  }

  /// v13 迁移：为老库补齐收支「其他」兜底分类（新库建库时已随预置写入）
  ///
  /// 按名字识别旧「其他」打标（用户可能建过同名分类，一并标记保持
  /// 兜底语义）；没有则新插入（图标/颜色与预置一致、排到同类型末位）。
  /// 幂等：已标记的分类跳过，半迁移中断后重跑不重复插入
  Future<void> _seedFallbackCategories() async {
    for (final type in BillType.values) {
      final roots = await (select(categories)
            ..where((c) => c.type.equalsValue(type))
            ..where((c) => c.parentId.isNull()))
          .get();
      final byName = roots.where((c) => c.name.trim() == '其他').toList();
      if (byName.isNotEmpty) {
        for (final c in byName.where((c) => !c.isFallback)) {
          await (update(categories)..where((x) => x.id.equals(c.id))).write(
            const CategoriesCompanion(isFallback: Value(true)),
          );
        }
        continue;
      }
      // 同类型无「其他」：排到末位（取现有最大 sortOrder + 1）
      var nextSort = 0;
      for (final c in roots) {
        if (c.sortOrder >= nextSort) nextSort = c.sortOrder + 1;
      }
      await into(categories).insert(
        CategoriesCompanion.insert(
          name: '其他',
          iconCode: Icons.more_horiz.codePoint,
          colorValue: 0xFFA8A8A8,
          type: type,
          isFallback: const Value(true),
          sortOrder: Value(nextSort),
        ),
      );
    }
  }
```

```
  /// 列存在才补差量的安全加列：列已在（半迁移中间态）时跳过
  Future<void> _ensureColumn(
    Migrator m,
    TableInfo table,
    GeneratedColumn column,
  ) async {
    final rows = await customSelect(
      'PRAGMA table_info([${table.actualTableName}])',
    ).get();
    final exists = rows.any((r) => r.data['name'] == column.$name);
    if (!exists) {
      await m.addColumn(table, column);
    }
  }

  /// 判断表是否已存在（迁移幂等检查用）
  Future<bool> _tableExists(String name) async {
    final rows = await customSelect(
      "SELECT 1 FROM sqlite_master WHERE type = 'table' AND name = ?",
      variables: [Variable(name)],
    ).get();
    return rows.isNotEmpty;
  }

  /// 列存在才补差量的原生 SQL 加列：列已在（半迁移中间态）时跳过。
  /// 供已从 drift 表定义移除的历史列使用（v10 借据多图改造），
  /// drift 不再认识这些列，只能走原生 DDL
  Future<void> _ensureRawColumn(
    String table,
    String name,
    String definition,
  ) async {
    final rows = await customSelect('PRAGMA table_info([$table])').get();
    final exists = rows.any((r) => r.data['name'] == name);
    if (!exists) {
      await customStatement('ALTER TABLE $table ADD COLUMN $definition');
    }
  }

  /// v7 迁移：旧资产分类名映射到钱迹式新分类体系
  ///
  /// 持久化只存分类名（无分类 ID），分类体系重构后旧数据按 kind 分别
  /// 映射：资产侧旧名 → 新名；负债侧"借款/其他"统一并入"其它负债"
  /// 承接（房贷/车贷新名不变，无需处理）。
  Future<void> _migrateLegacyAssetCategories() async {
    const assetRenames = <String, String>{
      '现金存款': '现金',
      '金融理财': '其它理财',
      '其他': '其它',
    };
```

```
    for (final entry in assetRenames.entries) {
      await customUpdate(
        "UPDATE assets SET category = ? WHERE category = ? AND kind = 0",
        variables: [Variable(entry.value), Variable(entry.key)],
      );
    }
    await customUpdate(
      "UPDATE assets SET category = '其它负债' WHERE category IN ('借款', '其他') AND kind = 1",
    );
  }
}

/// 各类历史备份文件的文件名前缀（与 BackupService 的类型识别约定一致）
const _kUpgradeBackupPrefix = 'chestnut_backup_v';
const _kDowngradeBackupPrefix = 'chestnut_downgrade_';

/// 数据库连接：后台线程执行 SQLite 操作，避免阻塞 UI 线程
///
/// 打开前先做版本预检（[schemaVersion] 由调用方传入）：升级与降级
/// 都先生成旧库快照，再交给 drift 正常打开。
LazyDatabase _openConnection(int schemaVersion) {
  return LazyDatabase(() async {
    final dir = await getApplicationDocumentsDirectory();
    final file = File(p.join(dir.path, kDatabaseFileName));
    await _backupBeforeOpen(file, schemaVersion);
    return NativeDatabase.createInBackground(file);
  });
}

/// drift 打开连接前的版本预检与快照备份
///
/// 此时库文件尚未被 drift 占用，VACUUM INTO 不受迁移事务限制：
/// · fileVersion < 代码版本：正常升级，备份旧库（保留最近 2 份），
///   迁移翻车时可回退；
/// · fileVersion > 代码版本：降级（装回旧版 App），高版本数据留底
///   后由 onUpgrade 的降级分支重建空库。
Future<void> _backupBeforeOpen(File file, int schemaVersion) async {
  if (!await file.exists()) return;
  Database? raw;
  try {
    raw = sqlite3.open(file.path);
    final fileVersion = raw.userVersion;
    // 0 = 未初始化的新文件，交给 drift 的 onCreate
    if (fileVersion == schemaVersion || fileVersion == 0) return;
    final dir = await getApplicationDocumentsDirectory();
    final stamp = DateFormat('yyyyMMdd_HHmmss').format(DateTime.now());
    final name = fileVersion < schemaVersion
        ? '$_kUpgradeBackupPrefix${fileVersion}_$stamp.sqlite'
        : '${_kDowngradeBackupPrefix}v${fileVersion}_$stamp.sqlite';
    raw.execute("VACUUM INTO '${p.join(dir.path, name)}'");
```

```
    // 升级前备份：正常升级保留最近 2 份
    pruneBackupsByPrefix(dir, prefix: _kUpgradeBackupPrefix);
    // 降级留底：同样只保留最近 2 份。降级是回退场景，反复装旧版
    // 会让快照无限堆积，限量兜底
    pruneBackupsByPrefix(dir, prefix: _kDowngradeBackupPrefix);
  } catch (_) {
    // 预检/备份失败放行：升级有事务保护，降级仍有重建兜底
  } finally {
    raw?.dispose();
  }
}

/// 按 [prefix] 清理历史备份，只保留最近 [keep] 份
/// （文件名含时间戳，字典序即时间序）。公开供钱迹导入留底复用。
void pruneBackupsByPrefix(Directory dir, {required String prefix, int keep = 2}) {
  final files =
      dir
          .listSync()
          .whereType<File>()
          .where((f) => p.basename(f.path).startsWith(prefix))
          .toList()
        ..sort((a, b) => a.path.compareTo(b.path));
  if (files.length <= keep) return;
  for (final f in files.take(files.length - keep)) {
    try {
      f.deleteSync();
    } catch (_) {
      // 单个清理失败忽略
    }
  }
}

/// 内置分类种子定义
class _CategorySeed {
  const _CategorySeed(
    this.name,
    this.icon,
    this.color,
    this.type,
    this.sortOrder, {
    this.children = const [],
    this.isFallback = false,
  });

  final String name;
  final IconData icon;
  final int color;
  final BillType type;
  final int sortOrder;
```

```
  /// 子分类定义（颜色跟随父分类，仅在预置时使用）
  final List<_CategorySeed> children;

  /// 兜底标记：仅「其他」为 true（分类删除迁移与钱迹导入的承接分类）
  final bool isFallback;
}

const _defaultCategories = <_CategorySeed>[
  // 支出分类：名称与图标均与编辑页图标库一一对应，子分类按高频排序
  _CategorySeed(   '餐饮',     Icons.restaurant,     0xFFFF9F43,     BillType.expense,     0,
    children: [      _CategorySeed(         '早餐',          Icons.free_breakfast,          0xFFFF9F43,          B
      _CategorySeed('午餐', Icons.rice_bowl, 0xFFFF9F43, BillType.expense, 1),
      _CategorySeed('晚餐', Icons.ramen_dining, 0xFFFF9F43, BillType.expense, 2),
      _CategorySeed(
        '买菜',
        Icons.shopping_basket,
        0xFFFF9F43,
        BillType.expense,
        3,
      ),
      _CategorySeed(
        '外卖',
        Icons.delivery_dining,
        0xFFFF9F43,
        BillType.expense,
        4,
      ),
      _CategorySeed('快餐', Icons.lunch_dining, 0xFFFF9F43, BillType.expense, 5),
      _CategorySeed('零食', Icons.icecream, 0xFFFF9F43, BillType.expense, 6),
      _CategorySeed('饮料', Icons.local_drink, 0xFFFF9F43, BillType.expense, 7),
      _CategorySeed(
        '下馆子',
        Icons.dinner_dining,
        0xFFFF9F43,
        BillType.expense,
        8,
      ),
    ],
  ),
  _CategorySeed(
    '交通',
    Icons.directions_subway,
    0xFF54A0FF,
    BillType.expense,
    1,
    children: [
      _CategorySeed(
        '公交',
        Icons.directions_bus,
        0xFF54A0FF,
```

```
        BillType.expense,
        0,
      ),
      _CategorySeed(
        '地铁',
        Icons.directions_subway,
        0xFF54A0FF,
        BillType.expense,
        1,
      ),
      _CategorySeed('打车', Icons.local_taxi, 0xFF54A0FF, BillType.expense, 2),
      _CategorySeed('停车', Icons.local_parking, 0xFF54A0FF, BillType.expense, 3),
      _CategorySeed(
        '加油',
        Icons.local_gas_station,
        0xFF54A0FF,
        BillType.expense,
        4,
      ),
      _CategorySeed('充电', Icons.ev_station, 0xFF54A0FF, BillType.expense, 5),
      _CategorySeed(
        '洗车',
        Icons.local_car_wash,
        0xFF54A0FF,
        BillType.expense,
        6,
      ),
      _CategorySeed('高速费', Icons.toll, 0xFF54A0FF, BillType.expense, 7),
      _CategorySeed('保养', Icons.build_circle, 0xFF54A0FF, BillType.expense, 8),
      _CategorySeed('单车', Icons.pedal_bike, 0xFF54A0FF, BillType.expense, 9),
      _CategorySeed('火车', Icons.train, 0xFF54A0FF, BillType.expense, 10),
      _CategorySeed('飞机', Icons.flight, 0xFF54A0FF, BillType.expense, 11),
    ],
  ),
  _CategorySeed(
    '购物',
    Icons.shopping_bag,
    0xFFFF6B81,
    BillType.expense,
    2,
    children: [
      _CategorySeed(
        '日用品',
        Icons.shopping_basket,
        0xFFFF6B81,
        BillType.expense,
        0,
      ),
      _CategorySeed('服饰', Icons.checkroom, 0xFFFF6B81, BillType.expense, 1),
      _CategorySeed('数码', Icons.devices, 0xFFFF6B81, BillType.expense, 2),
```

```
      _CategorySeed('电器', Icons.tv, 0xFFFF6B81, BillType.expense, 3),
    ],
  ),
  _CategorySeed(
    '住房',
    Icons.home,
    0xFF8D6E63,
    BillType.expense,
    3,
    children: [
      _CategorySeed('房租', Icons.home, 0xFF8D6E63, BillType.expense, 0),
      _CategorySeed('物业', Icons.home_work, 0xFF8D6E63, BillType.expense, 1),
      _CategorySeed('电费', Icons.bolt, 0xFF8D6E63, BillType.expense, 2),
      _CategorySeed('水费', Icons.water_drop, 0xFF8D6E63, BillType.expense, 3),
      _CategorySeed(
        '天燃气',
        Icons.local_fire_department,
        0xFF8D6E63,
        BillType.expense,
        4,
      ),
      _CategorySeed(
        '房贷',
        Icons.real_estate_agent,
        0xFF8D6E63,
        BillType.expense,
        5,
      ),
    ],
  ),
  _CategorySeed(
    '日常',
    Icons.wb_sunny,
    0xFF0984E3,
    BillType.expense,
    4,
    children: [
      _CategorySeed('理发', Icons.content_cut, 0xFF0984E3, BillType.expense, 0),
      _CategorySeed('话费', Icons.smartphone, 0xFF0984E3, BillType.expense, 1),
      _CategorySeed(
        '快递',
        Icons.local_shipping,
        0xFF0984E3,
        BillType.expense,
        2,
      ),
      _CategorySeed('网费', Icons.wifi, 0xFF0984E3, BillType.expense, 3),
      _CategorySeed(
        '会员订阅',
        Icons.subscriptions,
```

```
        0xFF0984E3,
        BillType.expense,
        4,
      ),
    ],
  ),
  _CategorySeed(
    '娱乐',
    Icons.sports_esports,
    0xFF9C88FF,
    BillType.expense,
    5,
    children: [
      _CategorySeed('住宿', Icons.hotel, 0xFF9C88FF, BillType.expense, 0),
      _CategorySeed('景点', Icons.attractions, 0xFF9C88FF, BillType.expense, 1),
      _CategorySeed(
        '游戏',
        Icons.sports_esports,
        0xFF9C88FF,
        BillType.expense,
        2,
      ),
      _CategorySeed(
        '演出',
        Icons.theater_comedy,
        0xFF9C88FF,
        BillType.expense,
        3,
      ),
      _CategorySeed('电影', Icons.theaters, 0xFF9C88FF, BillType.expense, 4),
    ],
  ),
  _CategorySeed(
    '医疗',
    Icons.medical_services,
    0xFF4CD7D0,
    BillType.expense,
    6,
    children: [
      _CategorySeed('药品', Icons.medication, 0xFF4CD7D0, BillType.expense, 0),
      _CategorySeed(
        '医疗',
        Icons.medical_services,
        0xFF4CD7D0,
        BillType.expense,
        1,
      ),
      _CategorySeed(
        '体检',
        Icons.health_and_safety,
```

```
        0xFF4CD7D0,
        BillType.expense,
        2,
      ),
      _CategorySeed(
        '门诊',
        Icons.local_hospital,
        0xFF4CD7D0,
        BillType.expense,
        3,
      ),
    ],
  ),
  _CategorySeed(
    '人情',
    Icons.card_giftcard,
    0xFFFF8FAB,
    BillType.expense,
    7,
    children: [
      _CategorySeed(
        '孝敬',
        Icons.volunteer_activism,
        0xFFFF8FAB,
        BillType.expense,
        0,
      ),
      _CategorySeed(
        '份子钱',
        Icons.connect_without_contact,
        0xFFFF8FAB,
        BillType.expense,
        1,
      ),
      _CategorySeed('礼物', Icons.redeem, 0xFFFF8FAB, BillType.expense, 2),
      _CategorySeed('压岁钱', Icons.currency_yen, 0xFFFF8FAB, BillType.expense, 3),
    ],
  ),
  _CategorySeed(
    '教育',
    Icons.school,
    0xFFFDCB6E,
    BillType.expense,
    8,
    children: [
      _CategorySeed('学费', Icons.school, 0xFFFDCB6E, BillType.expense, 0),
      _CategorySeed('书本', Icons.menu_book, 0xFFFDCB6E, BillType.expense, 1),
      _CategorySeed(
        '培训',
        Icons.cast_for_education,
```

```
        0xFFFDCB6E,
        BillType.expense,
        2,
      ),
      _CategorySeed('文具', Icons.edit, 0xFFFDCB6E, BillType.expense, 3),
      _CategorySeed('网课', Icons.language, 0xFFFDCB6E, BillType.expense, 4),
    ],
  ),
  // 兜底组：归不进前述分类的支出放这里，常驻末位（无子分类）；
  // 分类删除迁移与钱迹导入的承接分类，禁删。
  // 捐赠类支出不预置（1.2.3 撤销），用户可经图标库"公益"组自建
  _CategorySeed('其他', Icons.more_horiz, 0xFFA8A8A8, BillType.expense, 9,
      isFallback: true),
  // 收入分类
  _CategorySeed('工资', Icons.work, 0xFF4E9E5F, BillType.income, 0),
  _CategorySeed('奖金', Icons.emoji_events, 0xFFF39C12, BillType.income, 1),
  _CategorySeed('理财', Icons.trending_up, 0xFF27AE60, BillType.income, 2),
  _CategorySeed('红包', Icons.currency_yen, 0xFFE74C3C, BillType.income, 3),
  _CategorySeed('其他', Icons.more_horiz, 0xFFA8A8A8, BillType.income, 4,
      isFallback: true),
];
// ============ 文件: lib/data/flow_entry.dart ============
import 'database.dart';

/// 流水条目：账单（收支/转账）与借条的统一展示载体
///
/// 首页明细与账户流水页共用——两类异构记录经本类归一后，
/// 按业务日期组织日分组、按发生时刻排序（无时刻回退录入时间）。
class FlowEntry {
  const FlowEntry({
    required this.date,
    required this.createdAt,
    this.bill,
    this.debt,
  }) : assert(bill != null || debt != null);

  /// 业务日期：账单发生时间 / 借条借款日期（决定日分组与显示）
  ///
  /// 账单精确到日（0 点），借条含时刻——日内比较一律用 [dayMinute]，
  /// 天序比较须先归一到自然日，不能直接比 DateTime
  final DateTime date;

  /// 创建时间：发生时刻缺失或相同时的排序依据
  final DateTime createdAt;

  final Bill? bill;
  final DebtNote? debt;

  /// 发生时刻的日内分钟数（0..1439）：账单取 timeMinute（导入与
  /// 记账时都会写入），借条取借款日期的时分；null = 账单未填时刻
```

```
  int? get dayMinute {
    final b = bill, d = debt;
    if (b != null) return b.timeMinute;
    if (d != null) return d.borrowedAt.hour * 60 + d.borrowedAt.minute;
    return null;
  }
}

/// 合并账单与借条为流水条目，按业务日期倒序。
///
/// 同一自然日内按发生时刻倒序（如 9:30 在 9:20 上方）；未填时刻的
/// 账单沉底，与 SQL 层 `timeMinute DESC` 的 NULLS LAST 行为一致；
/// 时刻缺失或相同时再按创建时间倒序（后建的记录在前）。
///
/// 不做去重——账单与借条各自独立，同一笔现实业务可能同时产生
/// 一条账单和一条借条，两条都应呈现
List<FlowEntry> buildFlowEntries(List<Bill> bills, List<DebtNote> debts) {
  return [
    for (final bill in bills)
      FlowEntry(date: bill.date, createdAt: bill.createdAt, bill: bill),
    for (final debt in debts)
      FlowEntry(
        date: debt.borrowedAt,
        createdAt: debt.createdAt,
        debt: debt,
      ),
  ]..sort((a, b) {
      // 借条 date 含时刻、账单 date 是当日 0 点，直接比较会让同日
      // 借条因时刻膨胀恒在账单上方——先归一到自然日
      final byDay = _dayKey(b.date).compareTo(_dayKey(a.date));
      if (byDay != 0) return byDay;
      final am = a.dayMinute, bm = b.dayMinute;
      if (am != bm) {
        if (am == null) return 1;
        if (bm == null) return -1;
        return bm.compareTo(am);
      }
      return b.createdAt.compareTo(a.createdAt);
    });
}

/// 自然日键：y*10000+m*100+d，消掉日内时刻
int _dayKey(DateTime t) => t.year * 10000 + t.month * 100 + t.day;
// ============ 文件: lib/data/repositories/asset_flow_view_repository.dart ============
import '../../models/enums.dart';
import '../../utils/stream_utils.dart';
import '../database.dart';
import '../flow_entry.dart';
import 'asset_repository.dart';
import 'bill_image_repository.dart';
```

```dart
import 'bill_repository.dart';
import 'category_repository.dart';
import 'debt_note_repository.dart';

/// 流水视图模型：首页明细与账户流水页的统一数据载体
///
/// UI 只订阅一层流即可拿到渲染所需的全部数据，不再自行串联
/// 账单/借条/分类/图片等多张表的流。
class FlowView {
  const FlowView({
    this.asset,
    required this.entries,
    required this.categories,
    required this.relatedAssets,
    required this.billIdsWithImages,
    required this.debtIdsWithPhotos,
  });

  /// 账户流水页的账户本体（含已归档——归档账户也要能看流水）；
  /// 首页明细无单一账户概念，恒为 null
  final Asset? asset;

  /// 账单 + 借条合并后的流水条目（已按业务日期倒序）
  final List<FlowEntry> entries;

  /// 分类字典：id → 分类（账单行名称/图标按 id 查）
  final Map<int, Category> categories;

  /// 关联账户字典：id → 账户（含已归档，金额下方小字按 id 查名）
  final Map<int, Asset> relatedAssets;

  /// 带图片的账单 id 集合（相机角标）
  final Set<int> billIdsWithImages;

  /// 带照片的借条 id 集合（相机角标）
  final Set<int> debtIdsWithPhotos;
}

/// 流水视图仓储：把多张表的流组装成一个 [FlowView]
///
/// 只做"读组装"，不承担任何写逻辑——账户余额仍走 AssetRepository、
/// 借条增删改仍走 DebtNoteRepository，职责互不重叠。
class AssetFlowViewRepository {
  AssetFlowViewRepository(this._db);

  final AppDatabase _db;

  // 各仓储无状态（仅持 _db），就地构造零成本
  late final AssetRepository _assets = AssetRepository(_db);
  late final BillRepository _bills = BillRepository(_db);
```

```
  late final DebtNoteRepository _debts = DebtNoteRepository(_db);
  late final CategoryRepository _cats = CategoryRepository(_db);
  late final BillImageRepository _billImages = BillImageRepository(_db);

  /// 账户流水视图：账户本体 + 关联账单（转出/转入双向匹配）+
  /// 关联借条 + 分类/账户字典 + 两类图片标记
  Stream<FlowView> watchAssetFlow(int assetId) {
    final bills = _bills.watchBillsOfAsset(assetId);
    // 一条流同时产出账户本体与全量账户字典：watchAll 含已归档，
    // 修掉归档账户进流水页标题/概览消失的问题
    final assetData = _assets.watchAll().map(
          (list) => (
            self: list.where((a) => a.id == assetId).firstOrNull,
            map: {for (final a in list) a.id: a},
          ),
        );
    return combineLatest([
      assetData,
      bills,
      _debts.watchOfAsset(assetId),
      _cats.watchAllCategories().map(_categoryMap),
      // 图片标记依赖账单集合：账单一变就换订新的 id 流
      bills.switchMapLatest(
        (list) => _billImages.watchBillIdsWithImages(
          list.map((b) => b.id).toList(),
        ),
      ),
      // 借条照片标记是全表流，UI 按 id 自行过滤
      _debts.watchNoteIdsWithPhotos(),
    ]).map(
      (v) {
        final assetInfo = v[0] as ({Asset? self, Map<int, Asset> map});
        return FlowView(
          asset: assetInfo.self,
          relatedAssets: assetInfo.map,
          entries: buildFlowEntries(
            v[1] as List<Bill>,
            v[2] as List<DebtNote>,
          ),
          categories: v[3] as Map<int, Category>,
          billIdsWithImages: v[4] as Set<int>,
          debtIdsWithPhotos: v[5] as Set<int>,
        );
      },
    );
  }

  /// 首页明细视图：按 [period] 范围聚合账单，借条按借款日期落入
  /// 同一范围；日分组与小计由 UI 基于 entries 计算
  Stream<FlowView> watchHomeFlow(HomePeriod period, DateTime month) {
```

```
  final bills = switch (period) {
    HomePeriod.month => _bills.watchBillsInMonth(month),
    HomePeriod.year => _bills.watchBillsInYear(month.year),
    HomePeriod.all => _bills.watchAllBills(),
  };
  return combineLatest([
    bills,
    _debts.watchAll().map(
          (list) =>
              list.where((d) => _debtInRange(d, period, month)).toList(),
        ),
    _cats.watchAllCategories().map(_categoryMap),
    _assets.watchAll().map(_assetMap),
    bills.switchMapLatest(
      (list) => _billImages.watchBillIdsWithImages(
        list.map((b) => b.id).toList(),
      ),
    ),
    _debts.watchNoteIdsWithPhotos(),
  ]).map(
    (v) => FlowView(
      entries: buildFlowEntries(
        v[0] as List<Bill>,
        v[1] as List<DebtNote>,
      ),
      categories: v[2] as Map<int, Category>,
      relatedAssets: v[3] as Map<int, Asset>,
      billIdsWithImages: v[4] as Set<int>,
      debtIdsWithPhotos: v[5] as Set<int>,
    ),
  );
}

/// 分类列表转 id 字典
Map<int, Category> _categoryMap(List<Category> list) =>
    {for (final c in list) c.id: c};

/// 账户列表转 id 字典
Map<int, Asset> _assetMap(List<Asset> list) =>
    {for (final a in list) a.id: a};

/// 借条是否落在首页当前查看范围内（按借款日期判断）
bool _debtInRange(DebtNote debt, HomePeriod period, DateTime month) =>
    switch (period) {
      HomePeriod.month =>
        debt.borrowedAt.year == month.year &&
            debt.borrowedAt.month == month.month,
      HomePeriod.year => debt.borrowedAt.year == month.year,
      HomePeriod.all => true,
    };
```

```
}
// ============ 文件: lib/data/repositories/asset_repository.dart ============
import 'package:drift/drift.dart';
import 'package:intl/intl.dart';

import '../../models/enums.dart';
import '../database.dart';
import '../tables/assets.dart';

/// 单日净值点：(日期 `yyyy-MM-dd`, 净值分)
typedef NetValuePoint = (String day, int netCents);

/// 余额不足：资产账户资金流出后余额不能为负
///
/// 统一在 [AssetRepository.applyAccountDelta] 收口抛出；调用方事务随之
/// 回滚，页面层 catch 后直接把 [toString] 的中文信息 toast 给用户
class InsufficientBalanceException implements Exception {
  InsufficientBalanceException(this.accountName);

  /// 流出资产的账户名
  final String accountName;

  @override
  String toString() => '「$accountName」余额不足';
}

/// 资产页汇总快照：净值卡 / 负债率卡 / 借入借出卡共用
///
/// 口径说明：
/// · 资产侧合计 = 未归档且计入总资产的账户市值（不含借条）；
/// · 借条按方向并入两侧——借出计入总资产、借入计入总负债；
/// · "全部"口径（含不计入统计的借条）仅供借入借出卡展示台账总量。
class AssetSummary {
  const AssetSummary({
    required this.assetCents,
    required this.liabilityCents,
    required this.assetCount,
    required this.liabilityCount,
    required this.lendOutCents,
    required this.borrowInCents,
    required this.lendOutAllCents,
    required this.borrowInAllCents,
  });

  /// 账户资产合计（分，不含借条）
  final int assetCents;

  /// 账户负债合计（分，恒为正数，不含借条）
  final int liabilityCents;
```

```
  /// 参与净值的资产账户数
  final int assetCount;

  /// 参与净值的负债账户数
  final int liabilityCount;

  /// 计入总资产的借出合计（分）
  final int lendOutCents;

  /// 计入总负债的借入合计（分）
  final int borrowInCents;

  /// 全部借出合计（分，含"不计入统计"的借条）
  final int lendOutAllCents;

  /// 全部借入合计（分，含"不计入统计"的借条）
  final int borrowInAllCents;

  /// 总资产 = 账户资产 + 借出
  int get totalAssetCents => assetCents + lendOutCents;

  /// 总负债 = 账户负债 + 借入
  int get totalLiabilityCents => liabilityCents + borrowInCents;

  /// 净值 = 总资产 □ 总负债（分，可为负）
  int get netCents => totalAssetCents - totalLiabilityCents;

  /// 负债率 = 总负债 / 总资产（0.0~1.0，总资产为 0 时记 0）
  double get liabilityRate {
    if (totalAssetCents <= 0) return 0;
    return (totalLiabilityCents / totalAssetCents).clamp(0.0, 1.0);
  }

  /// 不含借条的快照净值（净值曲线末点口径：曲线历史全部来自
  /// 资产快照，末点若混入借条会与历史点口径不一致出现跳变）
  int get snapshotNetCents => assetCents - liabilityCents;
}

/// 资产仓储：资产/负债的增改、归档与净值聚合
///
/// 快照原则：市值的每一次变化都落一条 [AssetSnapshots]（只增不删），
/// 净值曲线、涨跌基准都从快照推导，资产表只保存"当前值"。
class AssetRepository {
  AssetRepository(this._db);

  final AppDatabase _db;

  /// 监听未归档资产（资产在前、负债在后，同组按 sortOrder）
  Stream<List<Asset>> watchActive() {
    return (_db.select(_db.assets)
```

```
        ..where((a) => a.archived.equals(false))
        ..orderBy([
          (a) => OrderingTerm.asc(a.kind),
          (a) => OrderingTerm.asc(a.sortOrder),
          (a) => OrderingTerm.asc(a.id),
        ]))
      .watch();
}

/// 监听已归档资产（按归档时间倒序，最近归档的在前）
Stream<List<Asset>> watchArchived() {
  return (_db.select(_db.assets)
        ..where((a) => a.archived.equals(true))
        ..orderBy([(a) => OrderingTerm.desc(a.updatedAt)]))
      .watch();
}

/// 监听全部资产（含已归档）：账单条目"金额下方账户小字"按 id 查名用，
/// 归档账户照常显示名字，只有删除后才兜底"已删除账户"
Stream<List<Asset>> watchAll() {
  return (_db.select(_db.assets)
        ..orderBy([(a) => OrderingTerm.asc(a.id)]))
      .watch();
}

/// 监听资产页汇总（净值卡/负债率卡/借入借出卡共用一份数据源）
///
/// 资产表与借条表在一条 SQL 里分别聚合，任一表变化都会重新发射。
/// include_in_net=false 的账户与 include_in_total=false 的借条
/// 不参与"计入"口径，但计入"全部"口径供台账展示。
Stream<AssetSummary> watchSummary() {
  return _db
      .customSelect(
        '''
        SELECT
          (SELECT COALESCE(SUM(value_cents), 0) FROM assets
            WHERE archived = 0 AND include_in_net = 1 AND kind = 0
          ) AS asset_cents,
          (SELECT COALESCE(SUM(value_cents), 0) FROM assets
            WHERE archived = 0 AND include_in_net = 1 AND kind = 1
          ) AS liability_cents,
          (SELECT COUNT(*) FROM assets
            WHERE archived = 0 AND include_in_net = 1 AND kind = 0
          ) AS asset_count,
          (SELECT COUNT(*) FROM assets
            WHERE archived = 0 AND include_in_net = 1 AND kind = 1
          ) AS liability_count,
          (SELECT COALESCE(SUM(amount_cents), 0) FROM debt_notes
            WHERE direction = 0 AND include_in_total = 1
          ) AS lend_out_cents,
```

```
          (SELECT COALESCE(SUM(amount_cents), 0) FROM debt_notes
            WHERE direction = 1 AND include_in_total = 1
          ) AS borrow_in_cents,
          (SELECT COALESCE(SUM(amount_cents), 0) FROM debt_notes
            WHERE direction = 0
          ) AS lend_out_all_cents,
          (SELECT COALESCE(SUM(amount_cents), 0) FROM debt_notes
            WHERE direction = 1
          ) AS borrow_in_all_cents
        ''',
        readsFrom: {_db.assets, _db.debtNotes},
      )
      .watch()
      .map((rows) {
        final r = rows.first;
        return AssetSummary(
          assetCents: r.read<int>('asset_cents'),
          liabilityCents: r.read<int>('liability_cents'),
          assetCount: r.read<int>('asset_count'),
          liabilityCount: r.read<int>('liability_count'),
          lendOutCents: r.read<int>('lend_out_cents'),
          borrowInCents: r.read<int>('borrow_in_cents'),
          lendOutAllCents: r.read<int>('lend_out_all_cents'),
          borrowInAllCents: r.read<int>('borrow_in_all_cents'),
        );
      });
}

/// 监听历史净值序列（不含今天：当天净值随资产表实时变化，由 UI 以
/// 当前值补上最后一点，保证曲线末点与卡片数字始终一致）
///
/// 聚合口径：按日取每项资产当天最后一条快照，净值 = Σ资产 □ Σ负债；
/// 当日尚无快照的资产不计入（快照在创建/更新市值时落，天然满足）。
/// 已归档资产的历史快照保留参与聚合，避免曲线回溯断层。
/// 注意：借条无快照，曲线为纯资产快照口径。
Stream<List<NetValuePoint>> watchNetHistory() {
  final query =
      _db.select(_db.assetSnapshots).join([
        innerJoin(
          _db.assets,
          _db.assets.id.equalsExp(_db.assetSnapshots.assetId),
        ),
      ])
        ..orderBy([
          OrderingTerm.asc(_db.assetSnapshots.day),
          OrderingTerm.asc(_db.assetSnapshots.createdAt),
          OrderingTerm.asc(_db.assetSnapshots.id),
        ]);
  return query.watch().map((rows) {
    final todayKey = _dayKey(DateTime.now());
```

```
  }

  /// 图标键（退格）
  Widget _iconKey(IconData icon, VoidCallback onTap) {
    return _KeyWrapper(
      onTap: onTap,
      child: Icon(icon, size: 22, color: AppColors.textPrimary),
    );
  }

  /// 文字功能键（清空 / 再记 / 今）
  Widget _textKey(String label, VoidCallback onTap) {
    return _KeyWrapper(
      onTap: onTap,
      child: Text(
        label,
        style: const TextStyle(
          fontSize: 15,
          fontWeight: FontWeight.w500,
          color: AppColors.textSecondary,
        ),
      ),
    );
  }

  /// 保存键：主色实底圆角，视觉重心
  Widget _saveKey() {
    return Padding(
      padding: const EdgeInsets.all(6),
      child: Material(
        color: AppColors.primary,
        borderRadius: BorderRadius.circular(AppDimens.radiusControl),
        child: InkWell(
          onTap: onDone,
          borderRadius: BorderRadius.circular(AppDimens.radiusControl),
          child: const SizedBox(
            height: 40,
            child: Center(
              child: Text(
                '保存',
                style: TextStyle(
                  fontSize: 16,
                  fontWeight: FontWeight.w600,
                  color: Colors.white,
                ),
              ),
            ),
          ),
        ),
      ),
    );
```

```
    );
  }
}

/// 键盘按键的水波反馈容器
class _KeyWrapper extends StatelessWidget {
  const _KeyWrapper({required this.onTap, required this.child});

  final VoidCallback onTap;
  final Widget child;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onTap,
        child: SizedBox(height: 52, child: Center(child: child)),
      ),
    );
  }
}
// ============ 文件: lib/widgets/photo_viewer_page.dart ============
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:photo_view/photo_view.dart';
import 'package:photo_view/photo_view_gallery.dart';

/// 全屏图片查看页：黑底 + 双指缩放/拖动 + 左右滑动切换多张，右上角关闭。
/// 图片面板与详情弹窗的大图预览共用本组件
class PhotoViewerPage extends StatefulWidget {
  const PhotoViewerPage({
    super.key,
    required this.paths,
    this.initialIndex = 0,
    this.openHeroTag,
  });

  /// 可切换的全部图片（本地文件路径）
  final List<String> paths;

  /// 打开时定位到第几张
  final int initialIndex;

  /// 被点缩略图的 Hero 标签：打开/关闭时从缩略图位置飞出/飞回；
  /// null 不启用 Hero 动画
  final Object? openHeroTag;

  @override
```

```
  State<PhotoViewerPage> createState() => _PhotoViewerPageState();
}

class _PhotoViewerPageState extends State<PhotoViewerPage> {
  late int _index = widget.initialIndex.clamp(0, widget.paths.length - 1);
  late final PageController _controller = PageController(initialPage: _index);

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final multiple = widget.paths.length > 1;
    final gallery = PhotoViewGallery.builder(
      pageController: _controller,
      itemCount: widget.paths.length,
      backgroundDecoration: const BoxDecoration(color: Colors.black),
      onPageChanged: (i) => setState(() => _index = i),
      builder: (_, i) => PhotoViewGalleryPageOptions(
        imageProvider: FileImage(File(widget.paths[i])),
        minScale: PhotoViewComputedScale.contained,
        maxScale: PhotoViewComputedScale.covered * 2,
      ),
    );
    return Scaffold(
      backgroundColor: Colors.black,
      body: Stack(
        children: [
          Positioned.fill(
            child: widget.openHeroTag == null
                ? gallery
                : Hero(tag: widget.openHeroTag!, child: gallery),
          ),
          SafeArea(
            child: Align(
              alignment: Alignment.topRight,
              child: GestureDetector(
                onTap: () => Navigator.of(context).pop(),
                child: Container(
                  width: 32,
                  height: 32,
                  margin: const EdgeInsets.only(right: 12),
                  decoration: BoxDecoration(
                    color: Colors.black.withValues(alpha: 0.45),
                    shape: BoxShape.circle,
                  ),
                  child: const Icon(
```

```
                    Icons.close,
                    size: 20,
                    color: Colors.white,
                  ),
                ),
              ),
            ),
          ),
          // 页码放底部居中：不与关闭按钮挤一角，滑动切换时位置反馈醒目
          if (multiple)
            SafeArea(
              child: Align(
                alignment: Alignment.bottomCenter,
                child: Container(
                  margin: const EdgeInsets.only(bottom: 20),
                  padding: const EdgeInsets.symmetric(
                    horizontal: 12,
                    vertical: 5,
                  ),
                  decoration: BoxDecoration(
                    color: Colors.black.withValues(alpha: 0.45),
                    borderRadius: BorderRadius.circular(14),
                  ),
                  child: Text(
                    '${_index + 1}/${widget.paths.length}',
                    style: const TextStyle(fontSize: 13, color: Colors.white),
                  ),
                ),
              ),
            ),
        ],
      ),
    );
  }
}
// ============ 文件: lib/widgets/pie_chart_card.dart ============
import 'dart:math' as math;

import 'package:fl_chart/fl_chart.dart';
import 'package:flutter/material.dart';

import '../theme/app_theme.dart';
import 'section_card.dart';

/// 环形占比图卡片（共享组件，支持手指拖动旋转，钱迹式）
///
/// 统计页（分类占比）与分类统计详情页（子分类构成）共用同一实现，
/// 保证两处环形图的视觉与交互完全一致。
/// 扇区颜色由内置多彩色板按序自动分配（循环取色），保证相邻扇区
/// 颜色差异明显；[items] 为（名称，金额分），金额为 0 的项不绘制。
```

```
class PieChartCard extends StatefulWidget {
  const PieChartCard({
    super.key,
    required this.title,
    required this.items,
    required this.centerLabel,
    required this.centerValue,
    required this.centerValueColor,
    required this.emptyText,
    this.onSliceTap,
  });

  final String title;

  /// 扇区数据：(名称, 金额分)
  final List<(String, int)> items;

  /// 内置多彩色板：相邻扇区色相拉开，循环使用
  static const List<Color> palette = [
    Color(0xFFF45B69), // 红
    Color(0xFFFF9F43), // 橙
    Color(0xFF2ECC71), // 绿
    Color(0xFF3498DB), // 蓝
    Color(0xFF9B59B6), // 紫
    Color(0xFFFDCB6E), // 黄
    Color(0xFF1ABC9C), // 青
    Color(0xFFFD79A8), // 粉
    Color(0xFFE17055), // 棕
    Color(0xFF636E72), // 灰
  ];

  /// 环心上方小字（如"支出合计"/"合计"）
  final String centerLabel;

  /// 环心金额文本（不含 □ 符号，由内部统一加前缀）
  final String centerValue;

  /// 环心金额颜色（支出红 / 收入绿 / 分类色）
  final Color centerValueColor;

  /// 无数据时的空态文案
  final String emptyText;

  /// 扇区点击回调（参数为 [items] 的下标）：统计页钻取到对应分类视图；
  /// null 时点击无响应
  final void Function(int index)? onSliceTap;

  @override
  State<PieChartCard> createState() => _PieChartCardState();
}
```

```
class _PieChartCardState extends State<PieChartCard> {
  /// 饼图起始角（度）：跟随手指拖动旋转（fl_chart 默认 0 = 3 点钟方向顺时针）
  double _startAngle = 0;

  /// 上一次手指相对饼图中心的角度（度）
  double? _lastAngle;

  /// 手指相对饼图中心的角度：0 为 3 点钟方向，顺时针为正（与 fl_chart 一致）
  double _angleOf(Offset local) {
    final size = context.size!;
    final v = local - Offset(size.width / 2, size.height / 2);
    return math.atan2(v.dy, v.dx) * 180 / math.pi;
  }

  void _onDragStart(DragStartDetails d) =>
      _lastAngle = _angleOf(d.localPosition);

  void _onDragUpdate(DragUpdateDetails d) {
    if (_lastAngle == null) return;
    final a = _angleOf(d.localPosition);
    var delta = a - _lastAngle!;
    // 归一化到 (-180, 180]，防止跨越 ±180 界时突跳一整圈
    delta =
        math.atan2(
          math.sin(delta * math.pi / 180),
          math.cos(delta * math.pi / 180),
        ) *
        180 /
        math.pi;
    setState(() => _startAngle += delta);
    _lastAngle = a;
  }

  /// 点击扇区定位：把点击点换算为极角，再按各扇区扫过角度累加，
  /// 定位到 [PieChartCard.items] 的下标（与扇区绘制顺序一致）。
  /// 环心区域保留给汇总文字不响应。
  void _onTapUp(TapUpDetails d) {
    final size = context.size!;
    final v = d.localPosition - Offset(size.width / 2, size.height / 2);
    // 环形区域：内径 52、扇区外缘 92，各留容差
    final r = v.distance;
    if (r < 44 || r > 100) return;
    final a = math.atan2(v.dy, v.dx) * 180 / math.pi;
    var rel = (a - _startAngle) % 360;
    if (rel < 0) rel += 360;
    final total = widget.items.fold<int>(0, (sum, item) => sum + item.$2);
    var acc = 0.0;
    for (var i = 0; i < widget.items.length; i++) {
      final cents = widget.items[i].$2;
```

```
      if (cents <= 0) continue; // 金额为 0 的项不绘制扇区
      acc += cents / total * 360;
      if (rel < acc) {
        widget.onSliceTap!(i);
        return;
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final total = widget.items.fold<int>(0, (sum, item) => sum + item.$2);
    return SectionCard(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            widget.title,
            style: const TextStyle(
              fontSize: 15,
              fontWeight: FontWeight.w600,
              color: AppColors.textPrimary,
            ),
          ),
          const SizedBox(height: AppDimens.gapMd),
          if (total == 0)
            SizedBox(
              height: 180,
              child: Center(
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Icon(
                      Icons.donut_large_outlined,
                      size: 48,
                      color: AppColors.textSecondary.withValues(alpha: 0.45),
                    ),
                    const SizedBox(height: AppDimens.gapSm),
                    Text(
                      widget.emptyText,
                      style: const TextStyle(
                        color: AppColors.textSecondary,
                        fontSize: 13,
                      ),
                    ),
                  ],
                ),
              ),
            )
          else
```

```
GestureDetector(
  // 水平拖动旋转：与外层列表的竖直滚动手势各管一个方向，
  // 竞技不冲突，旋转始终灵敏（pan 全向会与列表滚动抢手势）
  onHorizontalDragStart: _onDragStart,
  onHorizontalDragUpdate: _onDragUpdate,
  // 扇区点击钻取（onSliceTap 非空时启用）
  onTapUp: widget.onSliceTap == null ? null : _onTapUp,
  behavior: HitTestBehavior.opaque,
  child: SizedBox(
    height: 240,
    child: Stack(
      alignment: Alignment.center,
      // 标签略超出饼图区域时允许画进卡片内边距，避免文字被裁
      clipBehavior: Clip.none,
      children: [
        PieChart(
          PieChartData(
            startDegreeOffset: _startAngle,
            sections: [
              for (var i = 0; i < widget.items.length; i++)
                if (widget.items[i].$2 > 0)
                  PieChartSectionData(
                    value: widget.items[i].$2.toDouble(),
                    color: PieChartCard.palette[i % 10],
                    radius: 40,
                    showTitle: false,
                  ),
            ],
            centerSpaceRadius: 52,
            sectionsSpace: 2,
            pieTouchData: PieTouchData(enabled: false),
          ),
        ),
        // 外部标签：引导线 + "名称 百分比"，小扇区也显示
        Positioned.fill(
          child: CustomPaint(
            painter: _PieLabelPainter(
              startAngle: _startAngle,
              items: [
                for (var i = 0; i < widget.items.length; i++)
                  if (widget.items[i].$2 > 0)
                    (
                      widget.items[i].$1,
                      PieChartCard.palette[i % 10],
                      widget.items[i].$2 / total,
                    ),
              ],
            ),
          ),
        ),
```

```
                    // 环心汇总：标签 + 总金额
                    Column(
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        Text(
                          widget.centerLabel,
                          style: const TextStyle(
                            fontSize: 12,
                            color: AppColors.textSecondary,
                          ),
                        ),
                        const SizedBox(height: 2),
                        Text(
                          '□${widget.centerValue}',
                          style: TextStyle(
                            fontSize: 18,
                            fontWeight: FontWeight.w700,
                            color: widget.centerValueColor,
                            fontFeatures: const [FontFeature.tabularFigures()],
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

/// 饼图外部标签绘制：引导线 + "名称 百分比"，文字颜色与扇区一致。
/// 小于 2% 的小扇区也显示；同一侧标签纵向重叠时自动下移错开。
class _PieLabelPainter extends CustomPainter {
  _PieLabelPainter({required this.startAngle, required this.items});

  /// 饼图起始角（度）：与 PieChartData.startAngle 保持一致，随旋转更新
  final double startAngle;

  /// (名称, 颜色, 占比)，顺序与 PieChart 扇区一致
  final List<(String, Color, double)> items;

  @override
  void paint(Canvas canvas, Size size) {
    const rOut = 96.0; // 引导线起点：扇区外缘 (52+40) + 4
    const lineLen = 10.0;
    const gap = 3.0;
    final center = Offset(size.width / 2, size.height / 2);
```

```
    final rightRects = <Rect>[];
    final leftRects = <Rect>[];

    var angle = startAngle;
    for (final (name, color, percent) in items) {
      final sweep = percent * 360;
      final mid = (angle + sweep / 2) * math.pi / 180;
      angle += sweep;

      final dir = Offset(math.cos(mid), math.sin(mid));
      final p1 = center + dir * rOut;
      final p2 = center + dir * (rOut + lineLen);
      // 右半圆标签向右延伸，左半圆向左
      final toRight = math.cos(mid) >= 0;

      final tp = TextPainter(
        text: TextSpan(
          text: '$name ${(percent * 100).toStringAsFixed(1)}%',
          style: TextStyle(fontSize: 10.5, color: color),
        ),
        textDirection: TextDirection.ltr,
      )..layout();

      // 文字锚点：引导线末端外侧，垂直居中；与同侧已放置标签交叠时下移错开
      var labelLeft = toRight ? p2.dx + gap : p2.dx - gap - tp.width;
      var labelTop = p2.dy - tp.height / 2;
      final placed = toRight ? rightRects : leftRects;
      for (final r in placed) {
        if (labelTop < r.bottom + 2 && labelTop + tp.height > r.top - 2) {
          labelTop = r.bottom + 2;
        }
      }
      placed.add(Rect.fromLTWH(labelLeft, labelTop, tp.width, tp.height));

      // 引导线：径向段 p1→p2 + 连接段 p2→文字边缘（错开后为斜线）
      final linePaint = Paint()
        ..color = color
        ..strokeWidth = 1;
      final elbow = Offset(
        toRight ? labelLeft - gap : labelLeft + tp.width + gap,
        labelTop + tp.height / 2,
      );
      canvas.drawLine(p1, p2, linePaint);
      canvas.drawLine(p2, elbow, linePaint);
      tp.paint(canvas, Offset(labelLeft, labelTop));
    }
  }

  @override
  bool shouldRepaint(covariant _PieLabelPainter old) =>
```

```
      old.items != items || old.startAngle != startAngle;
}
// ============ 文件: lib/widgets/pin_pad_input.dart ============
import 'dart:async';
import 'dart:math';

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../theme/app_colors.dart';

/// 六位数字九宫格输入组件：锁屏 / 设置 / 验证 / 重置四场景共用
///
/// 输满 6 位回调 [PinPadInput.onCompleted]：返回 true 视为通过（清空圆点继续），
/// 返回 false 视为校验失败（抖动 + 清空重输）。
/// 标题/副标题与阶段切换由使用方（页面）管理，组件只管输入与反馈
class PinPadInput extends StatefulWidget {
  const PinPadInput({
    super.key,
    required this.onCompleted,
    this.onBiometric,
    this.biometricIcon = Icons.fingerprint,
  });

  final Future<bool> Function(String pin) onCompleted;

  /// 底部指纹键回调：null 时该位置空占位（保持 0 居中对齐）
  final VoidCallback? onBiometric;

  /// 指纹键图标（面容设备显示面容图标）
  final IconData biometricIcon;

  @override
  State<PinPadInput> createState() => _PinPadInputState();
}

class _PinPadInputState extends State<PinPadInput>
    with SingleTickerProviderStateMixin {
  final List<int> _digits = [];

  late final AnimationController _shake = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 400),
  );

  Future<void> _press(int digit) async {
    if (_digits.length >= 6 || _shake.isAnimating) return;
    // 震动反馈 fire-and-forget，无需等待
    unawaited(HapticFeedback.lightImpact());
    setState(() => _digits.add(digit));
```

```
    if (_digits.length < 6) return;
    final ok = await widget.onCompleted(_digits.join());
    if (!mounted) return;
    if (ok) {
      setState(_digits.clear);
    } else {
      // 校验失败：抖动提示后清空重输
      _shake.forward(from: 0);
      await Future.delayed(const Duration(milliseconds: 450));
      if (mounted) setState(_digits.clear);
    }
  }

  void _backspace() {
    if (_digits.isEmpty || _shake.isAnimating) return;
    HapticFeedback.lightImpact();
    setState(() => _digits.removeLast());
  }

  @override
  void dispose() {
    _shake.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        // 圆点行：实心=已输入，抖动动画提示校验失败
        AnimatedBuilder(
          animation: _shake,
          builder: (context, child) {
            final dx = _shake.isAnimating
                ? sin(_shake.value * 3 * pi) * 8 * (1 - _shake.value)
                : 0.0;
            return Transform.translate(offset: Offset(dx, 0), child: child);
          },
          child: Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (var i = 0; i < 6; i++)
                Container(
                  width: 12,
                  height: 12,
                  margin: const EdgeInsets.symmetric(horizontal: 7),
                  decoration: BoxDecoration(
                    shape: BoxShape.circle,
                    color: i < _digits.length
```

```
                    ? AppColors.primary
                    : Colors.transparent,
                border: Border.all(
                  color: AppColors.primary,
                  width: 1.5,
                ),
              ),
            ),
          ],
        ),
      ),
      const SizedBox(height: 28),
      // 九宫格：3×4 布局，底行三键等分
      SizedBox(
          width: 264,
          child: Column(
            children: [
              // 行间距 16：键面之间留呼吸感，避免上下行挤在一起
              for (final row in const [
                [1, 2, 3],
                [4, 5, 6],
                [7, 8, 9],
              ]) ...[
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [for (final d in row) _key(d)],
                ),
                const SizedBox(height: 16),
              ],
              // 底行与上三行同用 spaceBetween：边缘键中心严格重合
              //  （Expanded 等分会让 0/退格相对 2/5/8、3/6/9 列偏内）；
              // 指纹缺席时以 64 空占位保持键位不变
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  widget.onBiometric != null
                      ? _iconKey(
                          widget.biometricIcon,
                          onTap: widget.onBiometric!,
                        )
                      : const SizedBox(width: 64, height: 64),
                  _key(0),
                  _iconKey(Icons.backspace_outlined, onTap: _backspace),
                ],
              ),
            ],
          ),
        ),
    ],
  );
```

```
  }

  Widget _key(int digit) {
    return GestureDetector(
      onTap: () => _press(digit),
      child: Container(
        width: 64,
        height: 64,
        alignment: Alignment.center,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color: AppColors.textSecondary.withValues(alpha: 0.06),
        ),
        child: Text(
          '$digit',
          style: const TextStyle(
            fontSize: 24,
            fontWeight: FontWeight.w500,
            color: AppColors.textPrimary,
          ),
        ),
      ),
    );
  }

  /// 底部圆键（指纹/退格）：与数字键同款 64 灰圆，内容为图标
  Widget _iconKey(IconData icon, {required VoidCallback onTap}) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        width: 64,
        height: 64,
        alignment: Alignment.center,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color: AppColors.textSecondary.withValues(alpha: 0.06),
        ),
        child: Icon(icon, size: 26, color: AppColors.textPrimary),
      ),
    );
  }
}
// ============ 文件: lib/widgets/section_card.dart ============
import 'package:flutter/material.dart';

import '../theme/app_dimens.dart';

/// 统一卡片容器
///
/// 全应用唯一的卡片样式：白底 + 统一圆角 + 极浅投影。
```

```
/// 各页面用它包裹内容区块，保证视觉层次一致。
class SectionCard extends StatelessWidget {
  const SectionCard({
    super.key,
    required this.child,
    this.padding = const EdgeInsets.all(AppDimens.cardPadding),
    this.margin = EdgeInsets.zero,
    this.radius = AppDimens.radiusCard,
    this.onTap,
  });

  final Widget child;
  final EdgeInsetsGeometry padding;
  final EdgeInsetsGeometry margin;

  /// 卡片圆角：默认全局规范值，紧凑小卡可传更小值
  final double radius;

  /// 传入时可点击（卡片整体水波反馈）
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final card = Container(
      margin: margin,
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(radius),
        boxShadow: [
          // 极浅投影：仅用于把卡片从米色背景上托起，不做明显立体感
          BoxShadow(
            color: Colors.black.withValues(alpha: 0.03),
            blurRadius: 10,
            offset: const Offset(0, 2),
          ),
        ],
      ),
      child: padding == EdgeInsets.zero
          ? child
          : Padding(padding: padding, child: child),
    );
    if (onTap == null) return card;
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(radius),
        child: card,
      ),
    );
```

```
  }
}
// ============ 文件: lib/widgets/tag_picker_sheet.dart ============
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../data/database.dart';
import '../data/repositories/tag_repository.dart';
import '../theme/app_colors.dart';
import '../theme/app_dimens.dart';
import '../pages/settings/tag_manage_page.dart';

/// 标签选择弹层（记一笔/编辑页共用）
///
/// 顶部已选标签胶囊（点取消）+ 全部标签网格（点选）+ 输入框建新标签
/// + 右上角"管理"进入独立标签管理页。选中状态即时回传调用方。
class TagPickerSheet extends StatefulWidget {
  const TagPickerSheet({
    super.key,
    required this.selectedIds,
    required this.allTags,
    required this.onChanged,
  });

  /// 当前已选标签 id
  final Set<int> selectedIds;

  /// 全部标签（由调用方缓存传入，避免每次打开弹层都查库）
  final List<Tag> allTags;

  /// 选中变化回调（新增/取消都会触发）
  final void Function(Set<int> ids) onChanged;

  @override
  State<TagPickerSheet> createState() => _TagPickerSheetState();
}

class _TagPickerSheetState extends State<TagPickerSheet> {
  late Set<int> _selected;

  @override
  void initState() {
    super.initState();
    _selected = Set<int>.from(widget.selectedIds);
  }

  /// 切换某个标签的选中状态
  void _toggle(int id) {
    setState(() {
      if (_selected.contains(id)) {
```

```
        _selected.remove(id);
      } else {
        _selected.add(id);
      }
    });
    widget.onChanged(Set<int>.from(_selected));
  }

  /// 进入标签管理页（返回时刷新缓存）
  Future<void> _openManage() async {
    await Navigator.push(
      context,
      MaterialPageRoute<void>(builder: (_) => const TagManagePage()),
    );
    if (!mounted) return;
    // 管理页可能增删改标签，重新拉取全部
    final tags = await context.read<TagRepository>().getTags();
    if (!mounted) return;
    setState(() {
      widget.allTags
        ..clear()
        ..addAll(tags);
      // 清理已被删除的标签 id
      _selected.removeWhere((id) => !tags.any((t) => t.id == id));
    });
    widget.onChanged(Set<int>.from(_selected));
  }

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: const BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.vertical(top: Radius.circular(16)),
      ),
      child: SafeArea(
        top: false,
        child: Padding(
          padding: const EdgeInsets.fromLTRB(
            AppDimens.pagePadding,
            16,
            AppDimens.pagePadding,
            AppDimens.pagePadding,
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              // 标题行：标签 + 管理按钮
              Row(
```

```
                children: [
                  const Text(
                    '标签',
                    style: TextStyle(
                      fontSize: 17,
                      fontWeight: FontWeight.w600,
                      color: AppColors.textPrimary,
                    ),
                  ),
                  const Spacer(),
                  TextButton(
                    onPressed: _openManage,
                    child: const Text('管理'),
                  ),
                ],
              ),
              const SizedBox(height: AppDimens.gapMd),
              // 单一标签列表：选中=实底白字，未选=白底彩框彩字
              // （不再单列"已选区"，避免同一个标签上下重复出现）
              if (widget.allTags.isNotEmpty)
                Wrap(
                  spacing: 8,
                  runSpacing: 8,
                  children: [
                    for (final t in widget.allTags)
                      _TagChip(
                        tag: t,
                        selected: _selected.contains(t.id),
                        onTap: () => _toggle(t.id),
                      ),
                  ],
                )
              else
                const Padding(
                  padding: EdgeInsets.symmetric(vertical: 16),
                  child: Text(
                    '暂无标签，点右上角"管理"创建',
                    style: TextStyle(
                      fontSize: 13,
                      color: AppColors.textSecondary,
                    ),
                  ),
                ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```
/// 标签胶囊：选中态实底白字，未选中态浅底+彩色边框
class _TagChip extends StatelessWidget {
  const _TagChip({
    required this.tag,
    required this.selected,
    required this.onTap,
  });

  final Tag tag;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final color = Color(tag.color);
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
        decoration: BoxDecoration(
          // 未选中=白底彩框（描边胶囊，图2 风格）；选中=实底白字；
          // 圆角与卡片一致（radiusCard），避免胶囊过于圆润
          color: selected ? color : Colors.white,
          borderRadius: BorderRadius.circular(AppDimens.radiusCard),
          border: Border.all(color: color),
        ),
        child: Text(
          tag.name,
          style: TextStyle(
            fontSize: 13,
            color: selected ? Colors.white : color,
            fontWeight: selected ? FontWeight.w500 : FontWeight.w400,
          ),
        ),
      ),
    );
  }
}
// ============ 文件: lib/widgets/update_dialog.dart ============
import 'dart:async';

import 'package:flutter/material.dart';

import '../pages/settings/changelog_page.dart';
import '../services/update_service.dart';
import '../theme/app_colors.dart';

/// 更新弹窗结果：installed = 已拉起系统安装器；goBackup = 用户选择先去备份，
/// 调用方负责跳转备份页；backgrounded = 转入后台下载（下载继续，弹窗关闭）；
```

```
/// dismissed = 忽略/取消
enum UpdateDialogResult { installed, goBackup, backgrounded, dismissed }

/// 弹出更新弹窗；弱更点遮罩关闭返回 null（等效 dismissed）
Future<UpdateDialogResult?> showUpdateDialog(
  BuildContext context,
  UpdateInfo info,
) {
  return showDialog<UpdateDialogResult>(
    context: context,
    // 强更不可点遮罩关闭；弱更随时可关——下载已交给系统 DownloadManager，
    // 关弹窗 = 转后台继续下，回来看进度/安装即可
    barrierDismissible: !info.forceUpdate,
    builder: (_) => _UpdateDialog(info: info),
  );
}

/// 更新流程状态：展示 → 下载中（进度）→ 失败（可重试）
enum _Phase { idle, downloading, error }

class _UpdateDialog extends StatefulWidget {
  const _UpdateDialog({required this.info});

  final UpdateInfo info;

  @override
  State<_UpdateDialog> createState() => _UpdateDialogState();
}

class _UpdateDialogState extends State<_UpdateDialog> {
  final _service = UpdateService();

  var _phase = _Phase.idle;
  var _received = 0;
  var _total = -1;
  String? _error;

  /// 进度轮询器：下载已托管给系统 DownloadManager，
  /// 弹窗只负责每 500ms 问一次系统"下到哪了"
  Timer? _pollTimer;

  UpdateInfo get _info => widget.info;

  @override
  void initState() {
    super.initState();
    _resumePendingIfNeeded();
  }

  @override
```

```
  void dispose() {
    _pollTimer?.cancel();
    super.dispose();
  }

  /// 弹窗打开时接管后台下载：存在同版本的待续下载（上次转后台/进程被杀）
  /// 就直接进入下载态恢复进度显示；版本不一致的旧任务由服务取消清退
  Future<void> _resumePendingIfNeeded() async {
    final resumed = await _service.reusePendingDownload(_info.versionCode);
    if (!resumed || !mounted) return;
    final snapshot = await _service.pollDownloadSnapshot();
    if (!mounted) return;
    if (snapshot == null || snapshot.isActive) {
      // 还在下（或刚被接管）：直接恢复进度显示
      if (snapshot != null) {
        setState(() {
          _phase = _Phase.downloading;
          _received = snapshot.received;
          _total = snapshot.total;
        });
        _beginPolling();
      }
      return;
    }
    // 完成态/失败态不走轮询，就地收尾
    if (snapshot.isSuccessful) {
      await _finishInstall();
    } else {
      setState(() {
        _phase = _Phase.error;
        _error = '下载中断了，点"重试"接着来';
      });
    }
  }

  /// 发起下载：任务交给系统下载器，本地只记录凭据并开始轮询进度。
  /// 失败就地转错误态（可重试）
  Future<void> _startDownload() async {
    setState(() {
      _phase = _Phase.downloading;
      _received = 0;
      // 清单一般会带大小；系统下载器入队后从查询里回填
      _total = _info.fileSize ?? -1;
      _error = null;
    });
    try {
      await _service.startDownload(_info);
    } on UpdateException catch (e) {
      if (!mounted) return;
      setState(() {
```

```
        _phase = _Phase.error;
        _error = e.message;
      });
      return;
    }
    if (!mounted) return;
    _beginPolling();
  }

  void _beginPolling() {
    _pollTimer?.cancel();
    _pollTimer = Timer.periodic(const Duration(milliseconds: 500), (_) {
      unawaited(_pollOnce());
    });
  }

  /// 轮询一次系统下载状态：进行中刷新进度；完成走校验安装；
  /// 失败转错误态；记录消失（被系统清理/外部取消）回到展示态
  Future<void> _pollOnce() async {
    final snapshot = await _service.pollDownloadSnapshot();
    if (!mounted) return;
    if (snapshot == null) {
      _pollTimer?.cancel();
      setState(() => _phase = _Phase.idle);
      return;
    }
    if (snapshot.isSuccessful) {
      _pollTimer?.cancel();
      await _finishInstall();
      return;
    }
    if (snapshot.isFailed) {
      _pollTimer?.cancel();
      setState(() {
        _phase = _Phase.error;
        _error = '下载中断了，点"重试"接着来（已下载部分系统会自动续传）';
      });
      return;
    }
    setState(() {
      _received = snapshot.received;
      _total = snapshot.total;
    });
  }

  /// 下载完成 → 校验 → 安装，校验/定位失败转错误态（可重试）
  Future<void> _finishInstall() async {
    setState(() {
      // -1 哨兵 = 校验阶段：进度条转不确定态，文案切"正在校验"
      _received = -1;
```

```
      _total = -1;
    });
    try {
      final file = await _service.consumeDownloadedApk();
      final launched = await _service.installApk(file);
      if (!mounted) return;
      if (launched) {
        // 已跳到系统安装界面，更新弹窗使命完成
        Navigator.of(context).pop(UpdateDialogResult.installed);
      } else {
        setState(() {
          _phase = _Phase.error;
          _error = '无法打开安装器，请在系统设置允许安装后重试';
        });
      }
    } on UpdateException catch (e) {
      if (!mounted) return;
      setState(() {
        _phase = _Phase.error;
        _error = e.message;
      });
    }
  }

  /// "以后再说"：记住该版本，本次启动不再自动弹出
  Future<void> _dismiss() async {
    await _service.markIgnored(_info.versionCode);
    if (mounted) {
      Navigator.of(context).pop(UpdateDialogResult.dismissed);
    }
  }

  /// "取消下载"：让系统删除任务与半途文件，回到展示态
  Future<void> _cancelDownload() async {
    _pollTimer?.cancel();
    await _service.cancelDownload();
    if (!mounted) return;
    setState(() {
      _phase = _Phase.idle;
      _received = 0;
      _total = -1;
    });
  }

  /// "后台下载"：关弹窗继续下（系统托管），通知栏可见进度，
  /// 下次打开更新弹窗自动恢复进度显示
  void _backgroundDownload() {
    _pollTimer?.cancel();
    Navigator.of(context).pop(UpdateDialogResult.backgrounded);
  }
```

```
/// "去备份"：关闭弹窗，由调用方跳转备份页（备份方式由用户在页内自选）。
/// 下载未开始时直接关；下载中关掉即转后台，互不影响
void _goBackup() {
  _pollTimer?.cancel();
  Navigator.of(context).pop(UpdateDialogResult.goBackup);
}

/// 0..1 进度；总大小未知（-1）时返回 null（走不确定进度条）
double? get _progress =>
    _total > 0 ? (_received / _total).clamp(0.0, 1.0) : null;

/// 体积按 10^6 进制显示（MB），与 Android 系统安装器、应用商店口径一致；
/// 若按 1024（MiB）显示，用户会看到下载 61.8、安装时变 64.8 的困惑
String _mb(int bytes) => (bytes / 1000000).toStringAsFixed(1);

@override
Widget build(BuildContext context) {
  // 强更禁止系统返回键关闭；弱更随时可关（关 = 下载转后台）
  final canPop = !_info.forceUpdate;
  return PopScope(
    canPop: canPop,
    child: Dialog(
      backgroundColor: Colors.white,
      insetPadding: const EdgeInsets.symmetric(horizontal: 36),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      child: Padding(
        padding: const EdgeInsets.fromLTRB(20, 22, 20, 18),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            _buildHeader(),
            const SizedBox(height: 16),
            _buildNotes(),
            // 更新前备份引导：数据无价，给用户一个顺手兜底的入口
            const Padding(
              padding: EdgeInsets.only(top: 10),
              child: Row(
                children: [
                  Icon(
                    Icons.info_outline,
                    size: 14,
                    color: AppColors.textSecondary,
                  ),
                  SizedBox(width: 5),
                  Text(
                    '更新前建议先备份数据',
                    style: TextStyle(
                      fontSize: 12,
```

```
                      color: AppColors.textSecondary,
                    ),
                  ),
                ],
              ),
            ),
            // 只在展示态给入口：下载/重试中途跳走会打断更新流程
            if (_phase == _Phase.idle) _buildFullLogLink(),
            if (_phase == _Phase.downloading) ...[
              const SizedBox(height: 16),
              _buildProgress(),
            ],
            if (_phase == _Phase.error) ...[
              const SizedBox(height: 12),
              _buildError(),
            ],
            const SizedBox(height: 18),
            _buildActions(),
          ],
        ),
      ),
    ),
  );
}

/// 头部：Logo + 发现新版本 + 版本与发布日期
Widget _buildHeader() {
  return Row(
    children: [
      Image.asset('assets/images/logo.png', width: 42, height: 42),
      const SizedBox(width: 12),
      Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              const Text(
                '发现新版本',
                style: TextStyle(
                  fontSize: 17,
                  fontWeight: FontWeight.w700,
                  color: AppColors.textPrimary,
                ),
              ),
              if (_info.forceUpdate) ...[
                const SizedBox(width: 8),
                Container(
                  padding:
                      const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
                  decoration: BoxDecoration(
```

```
                    color: AppColors.expense.withValues(alpha: 0.1),
                    borderRadius: BorderRadius.circular(4),
                  ),
                  child: const Text(
                    '重要',
                    style: TextStyle(fontSize: 10, color: AppColors.expense),
                  ),
                ),
              ],
            ],
          ),
          const SizedBox(height: 2),
          // 发布时间缺失/解析失败时不显示日期段，只留版本号
          Text(
            switch (_info.publishedAtLocalText) {
              null => 'v${_info.versionName}',
              final date => 'v${_info.versionName}  ·  $date',
            },
            style: const TextStyle(
              fontSize: 12,
              color: AppColors.textSecondary,
            ),
          ),
        ],
      ),
    ],
  );
}

/// 更新内容：清单没写条目时给一句兜底文案
Widget _buildNotes() {
  final notes = _info.releaseNotes.isEmpty
      ? const ['修复了若干问题，优化使用体验']
      : _info.releaseNotes;
  return ConstrainedBox(
    // 条目再多也不顶满屏幕：限高滚动
    constraints: const BoxConstraints(maxHeight: 200),
    child: SingleChildScrollView(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          for (final note in notes)
            Padding(
              padding: const EdgeInsets.only(bottom: 8),
              child: Row(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const Padding(
                    padding: EdgeInsets.only(top: 7, right: 8),
                    child: Icon(
```

```
                      Icons.circle,
                      size: 5,
                      color: AppColors.textSecondary,
                    ),
                  ),
                  Expanded(
                    child: Text(
                      note,
                      style: const TextStyle(
                        fontSize: 14,
                        height: 1.5,
                        color: AppColors.textPrimary,
                      ),
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
}

/// 下载进度：已知总大小显示确定进度+百分比，否则不确定进度条；
/// _received = -1 表示校验阶段，显示"正在校验"
Widget _buildProgress() {
  final value = _progress;
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      ClipRRect(
        borderRadius: BorderRadius.circular(3),
        child: LinearProgressIndicator(
          value: value,
          minHeight: 6,
          backgroundColor: AppColors.fill,
          valueColor: const AlwaysStoppedAnimation(AppColors.primary),
        ),
      ),
      const SizedBox(height: 8),
      Text(
        switch (value) {
          null when _received < 0 => '正在校验安装包…',
          null => '正在下载 · ${_mb(_received)} MB',
          final v => '正在下载 ${(v * 100).toStringAsFixed(0)}%'
              ' · ${_mb(_received)}/${_mb(_total)} MB',
        },
        style: const TextStyle(
          fontSize: 12,
```

```
            color: AppColors.textSecondary,
            fontFeatures: [FontFeature.tabularFigures()],
          ),
        ),
      ],
    );
  }

  Widget _buildError() {
    return Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Icon(Icons.error_outline, size: 16, color: AppColors.expense),
        const SizedBox(width: 6),
        Expanded(
          child: Text(
            _error ?? '下载失败，请重试',
            style: const TextStyle(fontSize: 13, color: AppColors.expense),
          ),
        ),
      ],
    );
  }

  /// 查看全部更新记录：跳应用内更新记录页。
  /// 先推页面再关弹窗：页面路由压在弹窗之上，弹窗随后关闭，
  /// 顺序反了会用已销毁的弹窗 context 跳转
  Widget _buildFullLogLink() {
    return Center(
      child: TextButton.icon(
        onPressed: () {
          final nav = Navigator.of(context);
          nav.push(
            MaterialPageRoute<void>(builder: (_) => const ChangelogPage()),
          );
          nav.pop();
        },
        icon: const Icon(Icons.history, size: 16),
        label: const Text('查看全部更新记录', style: TextStyle(fontSize: 13)),
        style: TextButton.styleFrom(
          foregroundColor: AppColors.textSecondary,
          visualDensity: VisualDensity.compact,
        ),
      ),
    );
  }

  /// 底部按钮：随状态在 更新/重试/取消/后台下载 之间切换
  Widget _buildActions() {
    switch (_phase) {
```

```
      case _Phase.downloading:
        // 弱更：可取消（删任务回展示态）也可转后台（关弹窗继续下）；
        // 强更必须完成更新流程，不给取消、不给转后台。
        // 遮罩/返回键对强更仍然锁死
        return _info.forceUpdate
            ? _filledButton('正在下载…', enabled: false)
            : Row(
                children: [
                  Expanded(
                    child: _subtleButton(
                      '取消下载',
                      onTap: () => unawaited(_cancelDownload()),
                    ),
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    child: _filledButton('后台下载', onTap: _backgroundDownload),
                  ),
                ],
              );
      case _Phase.error:
        return Row(
          children: [
            if (!_info.forceUpdate) ...[
              Expanded(child: _subtleButton('取消', onTap: _dismiss)),
              const SizedBox(width: 12),
            ],
            Expanded(
              child: _filledButton('重试', onTap: _startDownload),
            ),
          ],
        );
      case _Phase.idle:
        // 主按钮独占一行；次按钮行：去备份恒有，以后再说仅弱更有
        // （强更去备份关闭后下次启动仍会自动弹出，不丢提醒）
        return Column(
          children: [
            _filledButton('立即更新', onTap: _startDownload),
            const SizedBox(height: 10),
            Row(
              children: [
                Expanded(child: _subtleButton('去备份', onTap: _goBackup)),
                if (!_info.forceUpdate) ...[
                  const SizedBox(width: 12),
                  Expanded(
                    child: _subtleButton('以后再说', onTap: _dismiss),
                  ),
                ],
              ],
            ),
```

```
          ],
        );
    }
  }

  /// 主按钮：蓝底白字；置灰表示不可点
  Widget _filledButton(String text, {VoidCallback? onTap, bool enabled = true}) {
    return SizedBox(
      height: 42,
      child: FilledButton(
        onPressed: enabled ? onTap : null,
        style: FilledButton.styleFrom(
          backgroundColor: AppColors.primary,
          disabledBackgroundColor: AppColors.fill,
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        ),
        child: Text(
          text,
          style: TextStyle(
            fontSize: 15,
            fontWeight: FontWeight.w600,
            color: enabled ? Colors.white : AppColors.textSecondary,
          ),
        ),
      ),
    );
  }

  /// 次按钮：浅灰底深字（忽略/取消）
  Widget _subtleButton(String text, {required VoidCallback onTap}) {
    return SizedBox(
      height: 42,
      child: OutlinedButton(
        onPressed: onTap,
        style: OutlinedButton.styleFrom(
          backgroundColor: AppColors.fill,
          side: BorderSide.none,
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        ),
        child: Text(
          text,
          style: const TextStyle(
            fontSize: 15,
            color: AppColors.textPrimary,
          ),
        ),
      ),
    );
  }
}
```